出处说明：本文是论文《Sheared LLaMA: Accelerating Language Model Pre-training via Structured Pruning》（Mengzhou Xia, Tianyu Gao, Zhiyuan Zeng, Danqi Chen, 2024；ICLR 2024）的中文译本，原文链接：https://arxiv.org/abs/2310.06694

Sheared LLaMA：通过结构化剪枝加速语言模型预训练（Sheared LLaMA: Accelerating Language Model Pre-training via Structured Pruning）

Mengzhou Xia[1] **Tianyu Gao**[1] **Zhiyuan Zeng**[2] **Danqi Chen**[1]
[1] 普林斯顿大学语言与智能（Princeton Language and Intelligence, Princeton University）
[2] 清华大学计算机科学与技术系（Department of Computer Science and Technology, Tsinghua University）
{mengzhou,tianyug,danqic}@cs.princeton.edu zengzy20@mails.tsinghua.edu.cn

发表于 ICLR 2024；arXiv:2310.06694v2 [cs.CL]，2024 年 4 月 11 日

[1] 代码与模型：https://github.com/princeton-nlp/LLM-Shearing。常见问题见附录 G。

译者注：图不翻译、不重绘，下文用一句话说明图意。「译者注」为帮助初学者理解而加。

摘要（Abstract）

LLaMA（Touvron et al., 2023a;b）以及其他近期出现的中等规模大语言模型（large language models, LLMs）的流行，凸显了构建更小却仍强大的 LLM 的潜力。尽管如此，在数万亿 token 上从零训练此类模型的成本仍然很高。本文研究结构化剪枝（structured pruning），作为从预训练的更大模型出发开发更小 LLM 的有效手段。我们的方法采用两项关键技术：（1）**目标化结构化剪枝**（targeted structured pruning），通过端到端方式移除层、注意力头以及中间维与隐维，把更大模型剪到指定目标形状；（2）**动态批加载**（dynamic batch loading），根据不同领域上损失的变化，动态更新每个训练批中采样数据的组成。我们通过给出 Sheared-LLaMA 系列来证明方法有效：把 LLaMA2-7B 模型剪到 1.3B 与 2.7B 参数。Sheared-LLaMA 模型在广泛的下游任务与指令调优评估上，超过同等规模的最先进开源模型，如 Pythia、INCITE、OpenLLaMA 以及同期的 TinyLlama；而相对从零训练这类模型，仅需约 3% 的算力。这项工作有力表明：利用现有 LLM 做结构化剪枝，是构建有竞争力的小规模 LLM 远更划算的路径。

---

1 引言（Introduction）

大语言模型（LLMs）在广泛自然语言任务上表现极强，但训练需要海量算力（OpenAI, 2023; Anthropic, 2023）。因此，人们对构建强大中等规模模型兴趣日增，例如 LLaMA（Touvron et al., 2023a;b）、MPT（MosaicML, 2023）与 Falcon（Almazrouei et al., 2023），它们便于高效推理与微调。这些 LLM 有多种适合不同场景的规模，但即便是最小的十亿参数级模型，从零训练每一个也需要大多数机构难以承受的计算资源。本文希望回答：

> **我们能否利用现有预训练 LLM，产出更小、通用且有竞争力的 LLM，同时所用算力远少于从零训练？**

我们探索用**结构化剪枝**达成这一目标。剪枝常被视为压缩任务专用模型的方案（Han et al., 2016; Li et al., 2016; Lagunas et al., 2021; Xia et al., 2022; Kurtic et al., 2023）：去掉冗余参数并加速推理，而不牺牲任务表现。然

而对通用 LLM，相对原模型，剪枝不可避免带来性能下降（Frantar & Alistarh, 2023; Sun et al., 2023; Ma et al., 2023），尤其是在剪枝后没有投入大量算力时。本文把剪枝用作开发更小却仍有竞争力 LLM 的有效途径，相对从零训练仅需一小部分训练算力。

> **图 1**：Sheared-LLaMA-2.7B 超过一系列相近规模开源模型，且仅需 OpenLLaMA-3B-v2 预算的约 1/32（3%）即可达到相当表现。

我们识别出该问题中的两个关键技术挑战。第一，如何决定最终剪枝后的架构，使其既表现强又推理高效？现有面向 LLM 的结构化剪枝技术（Xia et al., 2022; Ma et al., 2023）并不指定目标结构，导致剪后模型在表现与推理速度上欠优（表 4 与附录 F.2）。第二，如何继续预训练剪后模型以达到期望表现？我们观察到：相对从零训练，用原始预训练数据训练时，不同领域损失下降速率失衡。这表明剪后模型对不同领域（例如 GitHub 与 C4）保留的知识量不同，简单沿用预训练领域比例会导致数据使用低效（图 4）。为解决这些问题，我们提出「**LLM-shearing**」算法，包含以下两个组件：

- 我们提出新颖的剪枝算法，称为**目标化结构化剪枝**：把源模型剪到指定目标架构。目标架构通过借鉴现有预训练模型的配置来确定。我们的剪枝方法在给定约束下，在源模型中搜索最大限度保持表现的子结构。
- 我们设计**动态批加载**算法：按各领域损失下降速率成比例地加载训练数据，从而高效使用数据并加速整体表现提升。

我们通过把 LLaMA2-7B（Touvron et al., 2023b）剪成两个更小 LLM——Sheared-LLaMA-1.3B 与 Sheared-LLaMA-2.7B——证明方法有效。尽管剪枝与继续预训练仅额外使用 500 亿 token（即 OpenLLaMA 预训练预算的 5%），Sheared-LLaMA-1.3B 与 2.7B 在 11 项代表性下游任务（图 1；常识、阅读理解与世界知识）以及开放生成的指令调优上，超过相近规模的流行 LLM，包括 Pythia（Biderman et al., 2023）、INCITE（TogetherAI, 2023b）与 OpenLLaMA（Geng & Liu, 2023）。此外，下游表现轨迹表明：用更多 token 进一步训练剪后模型会带来更大收益。尽管我们仅对最大 7B 参数模型做实验，LLM-shearing 算法高度可泛化，未来可扩展到任意规模的大语言模型。

> 译者注：本文核心不是「剪完就部署」，而是「剪出一个形状合理的小模型，再继续预训练把它养回来」；动态批加载解决的是各领域恢复速度不均的问题。

---

2 LLM-Shearing（LLM-SHEARING）

给定现有大模型 M_S（源模型），我们研究如何高效产出更小、更强的模型 M_T（目标模型）。我们将其视为两阶段过程：（1）把 M_S **剪枝**为 M_T——这减少参数量但不可避免带来性能下降；（2）用标准语言建模目标对 M_T **继续预训练**以达到目标表现。尽管近期多数工作（Xia et al., 2022; Ma et al., 2023）聚焦前一阶段，我们发现后一阶段对从结构化剪枝产出有竞争力的通用 LLM 至关重要。

2.1 目标化结构化剪枝（TARGETED STRUCTURED PRUNING）

结构化剪枝移除成组的模型参数，以压缩模型并加速推理。然而现有结构化剪枝方法常产生偏离流行架构的非常规配置。例如 CoFiPruning（Xia et al., 2022）产出非均匀层配置（例如各层头数不同），相对标准均匀层配置会带来推理开销（第 4.2 节）。

> **图 2**：目标化结构化剪枝产出指定形状的紧凑稠密模型。浅色表示被剪掉的子结构。掩码变量 z 控制子结构被剪掉（$z = 0$）或保留（$z = 1$）。

本文旨在把源模型剪到我们指定的任意目标配置。这一目标有挑战性，因为它要求对外科手术般地缩小 Transformer 架构中的所有维度——据我们所知，此前尚未在大语言模型上完成。我们借鉴现有预训练模型的配置作为目标架构，基于如下直觉：这些配置已被良好优化以平衡模型表达力与推理效率。例如，产出 2.7B 模型时，我们以 INCITE-Base-3B 架构（TogetherAI, 2023a）作为目标结构。

我们的方法在不同粒度的模型参数上学习一组剪枝掩码——从全局的层与隐维（跨所有层持续存在），到局部的注意力头与中间维。假设源模型 M_S 有 L_S 层，每层含一个多头注意力模块（MHA）与一个前馈网络（FFN）。M_S 的隐状态维度为 d_S，每个 MHA 有 H_S 个头，每个 FFN 的中间维为 m_S。我们引入如下掩码变量：

粒度	层	隐维	头	中间维
剪枝掩码	$z^{\text{layer}} \in \mathbb{R}^{L_S}$	$z^{\text{hidden}} \in \mathbb{R}^{d_S}$	$z^{\text{head}} \in \mathbb{R}^{H_S}$（$\times L_S$）	$z^{\text{int}} \in \mathbb{R}^{m_S}$（$\times L_S$）

每个掩码变量控制相关子结构被剪掉还是保留。例如，若对应 $z^{\text{layer}} = 0$，则移除该层。图 2 示意剪枝掩码如何控制被剪结构。

我们把剪枝表述为约束优化问题（Platt & Barr, 1987）：学习剪枝掩码，搜索匹配预先指定目标架构、同时最大化表现的子网络。[2] 遵循 ℓ_0 正则方法（Louizos et al., 2018），我们参数化剪枝掩码以建模硬混凝土分布（hard concrete distributions）。这些分布支撑在 $[0, 1]$ 上，但概率质量集中于 0 或 1，从而支持离散的剪掉/保留决策。先前工作通常控制目标稀疏度（Wang et al., 2020; Xia et al., 2022），我们则用一对拉格朗日乘子直接对剪后模型形状施加约束。例如，对目标头数 H_T（并用 L_T、d_T、m_T 分别表示目标层数、隐维与中间维），单层上的约束为：

$$\tilde{\mathcal{L}}^{\text{head}}(\lambda, \phi, z) = \lambda^{\text{head}} \cdot \Big(\sum z^{\text{head}} - H_T \Big) + \phi^{\text{head}} \cdot \Big(\sum z^{\text{head}} - H_T \Big)^2.$$

对其他子结构施加类似约束。总体地,我们通过极小极大目标联合优化模型权重与剪枝掩码 $\min_{\theta,z} \max_{\lambda,\phi} \mathcal{L}_{\text{prune}}(\theta, z, \lambda, \phi)$：

$$\mathcal{L}_{\text{prune}}(\theta, z, \lambda, \phi) = \mathcal{L}(\theta, z) + \sum_{j=1}^{L_S} \tilde{\mathcal{L}}_j^{\text{head}} + \sum_{j=1}^{L_S} \tilde{\mathcal{L}}_j^{\text{int}} + \tilde{\mathcal{L}}^{\text{layer}} + \tilde{\mathcal{L}}^{\text{hidden}},$$

其中 $\mathcal{L}(\theta, z)$ 是用掩码后模型权重计算的语言建模损失。该目标将产出具有目标形状的剪后模型。理想情况下，在大量数据上运行该剪枝算法可直接得到强紧凑模型。实践中剪枝阶段昂贵（相对标准 LM 训练大约慢 $5\times$），且我们发现学到的掩码往往收敛很快。因此我们仅为剪枝分配有限预算（见表 5）。剪枝后，我们通过保留各子结构中与掩码变量相关的最高分组件来最终确定剪后架构，并用语言建模目标继续预训练剪后模型。我们把第二阶段称为**继续预训练**（continued pre-training）。

[2] 算法更详细阐述见附录 A。

2.2 动态批加载（DYNAMIC BATCH LOADING）

在大量数据上继续预训练对恢复剪后模型表现至关重要。初步实验中我们观察到一个令人惊讶的发现：在现有预训练数据集 RedPajama（TogetherAI, 2023b；LLaMA 的复现预训练数据）上继续预训练我们的剪后模型时，各领域损失下降速率与从零预训练不同，这意味着数据使用低效。

更具体地，我们先对每个领域拟合缩放函数（Hoffmann et al., 2022；细节见附录 B）到 LLaMA2 系列模型上。用该函数预测：若在同样数据上从零训练，一个假想的 1.3B LLaMA2 模型的损失。再把这些估计的参考损失

与继续预训练后剪后模型的损失比较。图 4（左）显示：我们的模型在 GitHub 上损失优于参考损失，而在 C4 上显著差于参考损失。这表明：相对高熵、更大领域（如 C4），剪枝在低熵、较小领域（如 GitHub）保留了更多知识。简单复用原始预训练数据分布 [3] 会导致数据使用低效与更差下游表现——即便整体损失看似很低，如第 4.1 节所示。

受近期工作启发（Xie et al., 2023），我们提出**动态批加载**：一种根据损失在线调整领域比例的高效算法，目标是确保模型在各领域大致同时达到参考损失。算法如下。

问题设定。预训练数据由 k 个领域 $D_1, \dots, D_k$ 组成，每个领域有留出验证集 D_i^{val}。在训练步 t，比例 $w_t[i]$ 的数据来自领域 D_i。我们为每个领域设定参考验证损失 $\ell_{\text{ref}}(D_i)$，并训练剪后模型达到该参考损失。

动态批加载。完整算法见算法 1。概要地，每 m 步评估模型得到 D^{val} 上的验证损失 ℓ_t（第 t 步），并根据每个领域上 $\ell_{\text{ref}}[i]$ 与 $\ell_t[i]$ 的差 $\Delta_t(D_i)$ 更新 w_t。更新规则按 Xie et al. (2023) 为指数上升：

$$\alpha_t = w_{t-m} \cdot \exp(\Delta_t); \qquad w_t = \frac{\alpha_t}{\sum_i \alpha_t[i]}.$$

我们把动态批加载用于剪枝阶段与继续预训练阶段。对剪枝，用原始预训练数据的领域权重作为 w_0；对继续预训练，用剪枝阶段最终权重作为 w_0。动态批加载是在线方案，训练中调整数据比例，无需训练辅助模型；它利用验证集上的参考损失动态调权，相对标准训练开销极小。这与 Xie et al. (2023) 不同——后者需要复杂多阶段过程来训练参考模型与代理模型。

更广地，即便不知道参考模型的确切训练数据，动态批加载也能用 RedPajama 等开源预训练数据集，训练 LLM 以匹配任意参考模型的表现。

参考损失的选择。默认地，我们用拟合缩放函数预测的损失作为参考（记为 **scaling reference**）。我们也实验另一种变体：直接用源模型的领域验证损失作参考（记为 **source reference**）。附录 F.4 表明两种变体都表现良好，但 scaling reference 在下游结果上略优，尤其在数学与编码任务上。当没有不同规模的源模型系列时，source reference 是可行替代。

[3] LLaMA2 预训练数据未公开。我们在 LLaMA1 模型上做了同样分析并观察到类似现象，表明这是与具体预训练数据无关的普遍问题。

算法 1：动态批加载（Dynamic Batch Loading）

输入： k 个领域的训练数据 $D_1, \dots, D_k$，验证数据 $D_1^{\text{val}}, \dots, D_k^{\text{val}}$，初始数据加载权重 $w_0 \in \mathbb{R}^k$，参考损失 $\ell_{\text{ref}} \in \mathbb{R}^k$，LM 损失 $\mathcal{L}$ 或剪枝损失 $\mathcal{L}_{\text{prune}}$，训练步数 T，每 m 步评估一次，模型参数 θ（剪枝时为 θ, z, ϕ, λ）。

对 $t = 1, \dots, T$：- 若 $t \bmod m = 0$：计算各领域验证损失 $\ell_t[i]$；$\Delta_t[i] \leftarrow \max\{\ell_t[i] - \ell_{\text{ref}}[i], 0\}$；用 `UpdateWeight` 更新 w_t。- 按比例 w_t 从各领域采样一批 B；- 若在剪枝：用 $\mathcal{L}_{\text{prune}}$ 在 B 上更新 θ, z, ϕ, λ；否则用 $\mathcal{L}$ 更新 θ。

子程序 `UpdateWeight(w, Δ)`：$\alpha \leftarrow w \cdot \exp(\Delta)$；返回归一化后的 $w \leftarrow \alpha / \sum_i \alpha[i]$。

3 实验（EXPERIMENTS）

3.1 设置（SETUP）

模型配置。主实验全程以 LLaMA2-7B（Touvron et al., 2023b）为源模型。[4] 我们做结构化剪枝实验，把该模型压缩到两个更小目标规模——2.7B 与 1.3B 参数。我们与相近规模的强预训练语言模型比较，包括 OPT-1.3B（Zhang et al., 2022）、Pythia-1.4B（Biderman et al., 2023）、TinyLlama-1.1B（Zhang et al., 2024）、OPT-2.7B、Pythia-2.8B、INCITE-Base-3B（TogetherAI, 2023b）、OpenLLaMA-3B-v1 与 OpenLLaMA-3B-v2（Geng & Liu, 2023）。1.3B 与 2.7B 模型的目标架构分别采用 Pythia-1.4B 与 INCITE-Base-3B。表 8 汇总了这些模型的架构细节。

表 1：Sheared-LLaMA 与其他模型所用预训练数据摘要。

模型	预训练数据	#Tokens
LLaMA1	LLaMA data	1T
LLaMA2	Unknown	2T
OPT	OPT data[5]	300B
Pythia	The Pile	300B
INCITE-Base	RedPajama	800B
OpenLLaMA v1	RedPajama	1T
OpenLLaMA v2	OpenLLaMA data[6]	1T
TinyLlama	TinyLlama data[7]	3T
Sheared-LLaMA	RedPajama	50B

数据。由于 LLaMA2 训练数据未公开，我们用 RedPajama（TogetherAI, 2023b）——LLaMA1 模型（Touvron et al., 2023a）的复现预训练数据集——做剪枝与继续预训练。该数据集涵盖七个领域：CommonCrawl、C4、Github、Wikipedia、Books、ArXiv 与 StackExchange。我们为每个领域构建含 200 万 token（等价于 500 条长度 4096 的序列）的留出验证集。剪枝阶段分配 4 亿 token，继续预训练过程分配 500 亿 token。遵循 LLaMA2 惯例，序列长度保持 4096。表 1 汇总了我们的模型与基线所用预训练数据。

[4] LLaMA1 结果见附录 F.6，Pythia 结果见附录 F.5。
[5] OPT 数据含 BookCorpus、Stories、CCNews、the Pile、PushShift.io Reddit 等。
[6] OpenLLaMA v2 用 RefinedWeb、StarCoder 与部分 RedPajama 的混合预训练。
[7] TinyLlama 数据是 SlimPajama 与 StarCoder 数据的混合。

评估。我们用 lm-evaluation-harness 包（Gao et al., 2021）在广泛下游任务套件上评估：（1）遵循 Pythia 与 LLaMA2，报告 ARC easy（ARC-E；Clark et al., 2018）、LAMBADA（Paperno et al., 2016）、LogiQA（Liu et al., 2020）、PIQA（Bisk et al., 2020）、SciQ（Welbl et al., 2017）与 WinoGrande（Sakaguchi et al., 2021）的 0-shot 准确率；（2）报告 Open LLM Leaderboard（Beeching et al., 2023）所用任务：10-shot HellaSwag（Zellers et al., 2019）、25-shot ARC Challenge（ARC-C；Clark et al., 2018）与 5-shot MMLU（Hendrycks et al., 2021）；（3）报告 32-shot Natural Questions（NQ；Kwiatkowski et al., 2019）的 exact match，以度量模型中的事实知识。由于训练模型遵循指令已成为 LLM 的关键应用（Ouyang et al., 2022; Taori et al., 2023），我们在指令调优上评估，并在从 ShareGPT 数据集 [8] 采样的指令—回复对上微调基线与 Sheared-LLaMA。更多细节见附录 E。

[8] https://sharegpt.com/

3.2 Sheared-LLaMA 超过同等规模的 LM（SHEARED-LLAMA OUTPERFORMS LMS OF EQUIVALENT SIZES）

我们表明：Sheared-LLaMA 在标准 LLM 基准与指令调优上超过相近规模的现有 LLM，同时仅使用从零训练这些模型所需算力预算的一小部分。

下游任务。表 2 给出 Sheared-LLaMA 与相近规模预训练模型的零样本与少样本下游任务表现。即便剪枝与继续预训练预算有限（约 50B token），Sheared-LLaMA 仍超过在显著更大算力上预训练的现有模型。Sheared-LLaMA-1.3B 超过 OPT-1.3B、Pythia-1.4B（用 300B token 预训练）与 TinyLlama-1.1B（用 3T token 预训练）。Sheared-LLaMA-2.7B 超过 INCITE-Base-3B（在 800B RedPajama token 上预训练）、OpenLLaMA-3B-v1（在 1T RedPajama token 上预训练）与 OpenLLaMA-3B-v2（在来自 RedPajama、RefinedWeb 与 StarCoder 的 1T token 上训练）。最值得注意的是：Sheared-LLaMA-1.3B 超过 TinyLlama-1.1B, 尽管后者在 3T token 上预训练——比预训练 LLaMA2-7B 与我们剪枝过程合计所用数据还多。这表明结构化剪枝是训练更小规模 LLM 更样本高效的途径。

表 2：Sheared-LLaMA 在下游任务上超过同等规模公开模型。括号内为所用 shot 数，未注明则为 0-shot。标 † 的模型使用不同于 RedPajama 的训练数据（见表 1）。

常识与阅读理解

模型（训练 #tokens）	SciQ	PIQA	WinoGrande	ARC-E	ARC-C (25)	HellaSwag (10)
LLaMA2-7B (2T)†	93.7	78.1	69.3	76.4	53.0	78.6
OPT-1.3B (300B)†	84.3	71.7	59.6	57.0	29.7	54.5
Pythia-1.4B (300B)†	86.4	70.9	57.4	60.7	31.2	53.0
TinyLlama-1.1B (3T)†	88.9	73.3	58.8	55.3	30.1	60.3
Sheared-LLaMA-1.3B (50B)	87.3	73.4	57.9	61.5	33.5	60.7
OPT-2.7B (300B)†	85.8	73.7	60.8	60.8	34.0	61.5
Pythia-2.8B (300B)†	88.3	74.0	59.7	64.4	36.4	60.8
INCITE-Base-3B (800B)	90.7	74.6	63.5	67.7	40.2	64.8
Open-LLaMA-3B-v1 (1T)	91.3	73.7	61.5	67.6	39.6	62.6
Open-LLaMA-3B-v2 (1T)†	91.8	76.2	63.5	66.5	39.0	67.6
Sheared-LLaMA-2.7B (50B)	90.8	75.8	64.2	67.0	41.2	70.8

续：世界知识等

模型（训练 #tokens）	LogiQA	BoolQ (32)	LAMBADA	NQ (32)	MMLU (5)	Average
LLaMA2-7B (2T)†	30.7	82.1	28.8	73.9	46.6	64.6
OPT-1.3B (300B)†	26.9	57.5	58.0	6.9	24.7	48.2
Pythia-1.4B (300B)†	27.3	57.4	61.6	6.2	25.7	48.9
TinyLlama-1.1B (3T)†	26.3	60.9	58.8	12.1	25.5	50.0
Sheared-LLaMA-1.3B (50B)	26.9	64.0	61.0	9.6	25.7	**51.0**
OPT-2.7B (300B)†	26.0	63.4	63.6	10.1	25.9	51.4
Pythia-2.8B (300B)†	28.0	66.0	64.7	9.0	26.9	52.5
INCITE-Base-3B (800B)	27.7	65.9	65.3	14.9	27.0	54.7
Open-LLaMA-3B-v1 (1T)	28.4	70.0	65.4	18.6	27.0	55.1
Open-LLaMA-3B-v2 (1T)†	28.1	69.6	66.5	17.1	26.9	55.7
Sheared-LLaMA-2.7B (50B)	28.9	73.7	68.4	16.5	26.4	**56.7**

指令调优。如图 3 所示，经指令调优的 Sheared-LLaMA 相对所有相近规模其他预训练模型取得更高胜率。这表明我们的 2.7B 模型可作为指令调优的强基座，并有能力生成长、连贯且信息丰富的回复（示例见附录 E）。

图 3: Sheared-LLaMA 在指令调优上超过 Pythia-1.4B、INCITE-Base-3B、OpenLLaMA-3B-v1 与 OpenLLaMA-3B-v2（图中给出对战胜率百分比）。

4 分析（ANALYSIS）

4.1 动态批加载的有效性（EFFECTIVENESS OF DYNAMIC BATCH LOADING）

我们从三方面分析动态批加载的有效性：（1）各领域最终 LM 损失；（2）训练全程各领域数据用量；（3）下游任务表现。本节全部结果基于 Sheared-LLaMA-1.3B。[9]

各领域损失差异。动态批加载旨在平衡各领域损失下降速率，确保损失大致同时达到参考值。图 4 展示我们的模型损失（原始批加载与动态批加载）与参考损失之差；参考损失由拟合到假想 1.3B 参数 LLaMA2 模型的缩放函数估计。原始批加载导致各领域损失差差异很大：例如 GitHub 损失降到参考值以下，而 C4 损失落后。动态批加载则均匀降低损失，并使各领域损失差非常接近，表明数据使用更高效。

数据用量。表 3 比较 RedPajama 的数据比例与我们动态加载方法的数据用量（图 6 展示训练中领域权重如何变化）。可见动态批加载从 Book 与 C4 子集加载更多数据，表明这些领域对剪后模型更难恢复。

表 3：动态批加载相对原始比例的领域数据用量。

	CC	GitHub	Book	StackExchange	Wiki	ArXiv	C4
RedPajama（原始）	67.0%	4.5%	4.5%	2.0%	4.5%	2.5%	15.0%
Dynamic Batch Loading	36.1%	0.8%	9.1%	1.0%	3.1%	0.7%	49.2%

下游表现。如图 5 所示，用动态批加载训练的剪后模型，下游表现优于在原始 RedPajama 分布上训练。这表明动态批加载带来的更平衡损失下降，可转化为更强下游能力。

[9] 我们也实验了剪枝时启发式排除简单领域的做法，但发现继续预训练后损失不均问题仍存在。细节见附录 F.8。

图 4：剪后模型（1.3B）与估计参考损失之差：原始批加载 vs. 动态批加载。
图 5：Sheared-LLaMA-1.3B 在原始数据比例与动态批加载下的下游任务平均准确率随训练 token 数变化。

4.2 与其他剪枝方法的比较（COMPARISON TO OTHER PRUNING APPROACHES）

我们在验证困惑度（perplexity）上把 LLM-shearing 与其他剪枝方法比较；困惑度是整体模型能力的强指示（Xia et al., 2023）。

目标化剪后模型具有更高推理速度。先前工作如 CoFiPruning（Xia et al., 2022）产出结构化剪后模型，但这些模型常有非均匀层配置（例如各层头数不同）。这种跨层非均匀性因架构不规则会引入训练与推理开销。我们在相同 0.4B 剪枝预算与 RedPajama 数据比例下实验 CoFiPruning 与目标化结构化剪枝以便公平比较。表 4 显示：在相同稀疏度下，我们的目标化剪后模型相对非均匀剪后的 CoFiPruning 有更高推理速度，尽管困惑度略高。目标化结构化剪枝大约需要继续预训练多 0.5B token 才能匹配 CoFiPruning 的困惑度。但训练时这一次性额外算力是合理的，因为它带来对真实应用与有效实用至关重要的更高效模型架构。不同剪枝方法推理速度的更多细节见附录 F.9。

表 4：均匀层配置的目标化结构化剪枝，与非均匀层配置的 CoFiPruning：验证困惑度与推理生成速度（tokens/秒）。推理速度在 Nvidia A100（80G）GPU 上、单实例最多生成 512 token 时测量。

	Layer Config	PPL ↓	Speed ↑		Layer Config	PPL ↓	Speed ↑
1.3B CoFiPruning	非均匀	9.1	51	2.7B CoFiPruning	非均匀	7.0	37
Targeted pruning	均匀	10.3	58	Targeted pruning	均匀	7.7	43

与 LLM-Pruner（Ma et al., 2023）的比较。我们在附录 F.2 中把目标化结构化剪枝与近期结构化剪枝工作 LLM-Pruner 比较。我们表明：在相同算力预算、稀疏度与训练数据分布下，我们的剪后模型达到更低困惑度、更优化的架构与更快推理速度。

4.3 额外分析（ADDITIONAL ANALYSIS）

剪枝与继续预训练的预算分配。直觉上，把更多算力分给剪枝阶段有助于识别更好的子网络结构。我们在固定 5B token 总预算内，探索在剪枝与继续预训练阶段不同的数据分配。表 5 显示：控制总 token 量时，增大剪枝预算 consistently 改进困惑度。但由于剪枝比继续预训练更贵，我们决定给剪枝分配 0.4B token。训练吞吐细节见附录 C。

表 5：剪枝与继续预训练（CT）的数据预算分配及对应困惑度。

# Tokens Pruning	# Tokens CT	PPL Pruning	PPL CT
0.2B	4.6B	12.99	7.46
0.4B	4.4B	10.29	7.32
0.8B	4.0B	9.01	7.23
1.6B	3.2B	8.04	7.08

更多分析。附录提供进一步分析：（1）Sheared-LLaMA 在数学与编码上的评估（附录 F.3）；（2）Pythia 模型剪枝（附录 F.5）；（3）剪枝时排除简单领域的影响（附录 F.8）。

5 相关工作（RELATED WORK）

剪枝。结构化剪枝在计算机视觉与自然语言处理中作为模型压缩技术被广泛研究：任务专用模型（如分类模型）往往过参数化，可在对表现影响极小的情况下大幅剪枝（Han et al., 2016; Wen et al., 2016; Liu et al., 2017; Luo et al., 2017; Cai et al., 2019; Deng et al., 2020; Hou et al., 2020; Wang et al., 2020; Lagunas et al., 2021; Xia et al., 2022; Kurtic et al., 2023）。非结构化剪枝（Frankle & Carbin, 2018; Li et al., 2020; Chen et al., 2020; Sanh et al., 2020）剪掉单个神经元而非结构化块；尽管压缩率通常更高，但对模型加速并不实用。

在 LLM 时代，主流 NLP 流程已从任务专用模型转向通用 LM，冗余空间更小。非结构化剪枝、半结构化剪枝（Frantar & Alistarh, 2023; Sun et al., 2023）与结构化剪枝（Ma et al., 2023）即便在适中稀疏度下也会导致 LLM 显著性能下降。值得注意的是，先前工作都固定原模型或仅做极少调优。我们把剪枝视为初始化，并认为有必要投入大量算力继续预训练以恢复表现。

高效预训练方法。与我们的剪枝正交，有大量工作改进 LLM 训练效率。例如量化降低权重与激活的数值精度并加速训练与推理（Dettmers et al., 2022; 2023; Xiao et al., 2023）。知识蒸馏（Hinton et al., 2015; Sanh et al., 2019; Jiao et al., 2020; Sun et al., 2020）在更大模型的预测上训练更小模型，对任务专用模型有效（Xia et al., 2022）。对预训练 LLM，尽管从教师蒸馏在相同训练步数下可提升学生质量（Rae et al., 2021; Blakeney et al., 2022），但由于教师带来的超额推理成本，它不如剪枝加继续训练划算（Jha et al., 2023）。还有更多方法提升 LM 训练效率，如动态架构（Gong et al., 2019; Zhang & He, 2020）与高效优化器（Chen et al., 2023; Liu et al., 2023）。但如（Kaddour et al., 2023; Bartoldson et al., 2023）所示，承诺的训练效率收益未必 consistently 实现。

也有基于数据的方法提升训练效率。消除重复数据被发现有效（Lee et al., 2021）。多种批选择技术提议按更高损失（Jiang et al., 2019）或更大可约减损失（Mindermann et al., 2022）等标准优先选择数据。Xie et al. (2023) 提议通过训练代理模型估计各领域最优数据权重来优化数据混合。

---

6 讨论（DISCUSSION）

局限与未来工作。第一，该方法高度依赖开源预训练数据集与模型的可用性。若某领域未覆盖在预训练数据中，方法可能难以在该领域上良好恢复表现。第二，受计算限制，我们仅以 7B 参数模型作源模型实验。但我们的方法高度可泛化，未来研究可扩展到更大模型。

结论。本文提出结构化剪枝作为创建有竞争力更小规模 LLM 的高效方法。我们的两阶段方法结合目标化结构化剪枝与继续预训练，并引入动态批加载以提升预训练数据效率。我们用标准预训练所需算力的一小部分训练出一系列有竞争力的 Sheared-LLaMA 模型。结果表明：当有强大规模模型可用时，产出低成本小 LLM 有一条有前景的路径。随着更强 LLM 与更大预训练数据集出现，我们的方法可轻易扩展到这些进展，以创造更好的小模型。

致谢（ACKNOWLEDGEMENTS）

我们感谢 Sadhika Malladi、Tanya Goyal、Ofir Press、Adithya Bhaskar 以及 Princeton NLP 组审阅论文并提供有益反馈。我们也感谢 MosaicML 工程团队在使用 Composer 包实现细节上的宝贵帮助。Mengzhou Xia 获 Bloomberg Data Science Ph.D. Fellowship 资助，Tianyu Gao 获 IBM PhD Fellowship 资助。本研究亦获微软 Azure 积分通过“Accelerate Foundation Models Academic Research”计划支持。

参考文献（REFERENCES）

译者注：参考文献按原文保留英文。

Ebtesam Almazrouei, Hamza Alobeidli, Abdulaziz Alshamsi, Alessandro Cappelli, Ruxandra Cojocaru, Merouane Debbah, Etienne Goffinet, Daniel Heslow, Julien Launay, Quentin Malartic, Badreddine Noune, Baptiste Pannier, and Guilherme Penedo. Falcon-40B: an open large language model with state-of-the-art performance. 2023.

Anthropic. Introducing claude, 2023.

Brian R Bartoldson, Bhavya Kailkhura, and Davis Blalock. Compute-efficient deep learning: Algorithmic trends and opportunities. Journal of Machine Learning Research, 24:1–77, 2023.

Jason Baumgartner, Savvas Zannettou, Brian Keegan, Megan Squire, and Jeremy Blackburn. The pushshift reddit dataset. ArXiv, abs/2001.08435, 2020.

Edward Beeching, Clémentine Fourrier, Nathan Habib, Sheon Han, Nathan Lambert, Nazneen Rajani, Omar Sanseviero, Lewis Tunstall, and Thomas Wolf. Open llm leaderboard. https://huggingface.co/spaces/HuggingFaceH4/open_
2023.

Stella Biderman, Hailey Schoelkopf, Quentin Gregory Anthony, Herbie Bradley, Kyle O'Brien, Eric Hallahan, Mohammad Aflah Khan, Shivanshu Purohit, USVSN Sai Prashanth, Edward Raff, et al. Pythia: A suite for analyzing large language models across training and scaling. In International Conference on Machine Learning, pp. 2397–2430. PMLR, 2023.

Yonatan Bisk, Rowan Zellers, Jianfeng Gao, Yejin Choi, et al. Piqa: Reasoning about physical commonsense in natural language. In Proceedings of the AAAI conference on artificial intelligence, volume 34, pp. 7432–7439, 2020.

Cody Blakeney, Jessica Zosa Forde, Jonathan Frankle, Ziliang Zong, and Matthew L Leavitt. Reduce, reuse, recycle: Improving training efficiency with distillation. arXiv preprint arXiv:2211.00683, 2022.

Han Cai, Chuang Gan, Tianzhe Wang, Zhekai Zhang, and Song Han. Once-for-all: Train one network and specialize it for efficient deployment. In International Conference on Learning Representations, 2019.

Tianlong Chen, Jonathan Frankle, Shiyu Chang, Sijia Liu, Yang Zhang, Zhangyang Wang, and Michael Carbin. The lottery ticket hypothesis for pre-trained bert networks. In Advances in Neural Information Processing Systems, 2020.

Xiangning Chen, Chen Liang, Da Huang, Esteban Real, Kaiyuan Wang, Yao Liu, Hieu Pham, Xuanyi Dong, Thang Luong, Cho-Jui Hsieh, et al. Symbolic discovery of optimization algorithms. arXiv preprint arXiv:2302.06675, 2023.

Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try arc, the ai2 reasoning challenge. arXiv preprint arXiv:1803.05457, 2018.

Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. Advances in Neural Information Processing Systems, 35:16344–16359, 2022.

Lei Deng, Guoqi Li, Song Han, Luping Shi, and Yuan Xie. Model compression and hardware acceleration for neural networks: A comprehensive survey. Proceedings of the IEEE, 108(4):485–532, 2020.

Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. Llm.int8 (): 8-bit matrix multiplication for transformers at scale. arXiv preprint arXiv:2208.07339, 2022.

Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. Qlora: Efficient finetuning of quantized llms. arXiv preprint arXiv:2305.14314, 2023.

Yann Dubois, Xuechen Li, Rohan Taori, Tianyi Zhang, Ishaan Gulrajani, Jimmy Ba, Carlos Guestrin, Percy Liang, and Tatsunori B Hashimoto. Alpacafarm: A simulation framework for methods that learn from human feedback. arXiv preprint arXiv:2305.14387, 2023.

Jonathan Frankle and Michael Carbin. The lottery ticket hypothesis: Finding sparse, trainable neural networks. In International Conference on Learning Representations, 2018.

Elias Frantar and Dan Alistarh. Sparsegpt: Massive language models can be accurately pruned in one-shot, 2023. arXiv preprint arXiv:2301.00774, 2023.

Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, et al. The pile: An 800gb dataset of diverse text for language modeling. arXiv preprint arXiv:2101.00027, 2020.

Leo Gao, Jonathan Tow, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Kyle McDonell, Niklas Muennighoff, Jason Phang, Laria Reynolds, Eric Tang, Anish Thite, Ben Wang, Kevin Wang, and Andy Zou. A framework for few-shot language model evaluation, September 2021.

Xinyang Geng and Hao Liu. Openllama: An open reproduction of llama, May 2023.

Linyuan Gong, Di He, Zhuohan Li, Tao Qin, Liwei Wang, and Tieyan Liu. Efficient training of bert by progressively stacking. In International conference on machine learning, pp. 2337–2346. PMLR, 2019.

Kshitij Gupta, Benjamin Thérien, Adam Ibrahim, Mats L Richter, Quentin Anthony, Eugene Belilovsky, Irina Rish, and Timothée Lesort. Continual pre-training of large language models: How to (re) warm your model? arXiv preprint arXiv:2308.04014, 2023.

Felix Hamborg, Norman Meuschke, Corinna Breitinger, and Bela Gipp. news-please: A generic news crawler and extractor. In Proceedings of the 15th International Symposium of Information Science, pp. 218–223, 2017.

Song Han, Huizi Mao, Dally, and William Dally. Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding. In International Conference on Learning Representations, 2016.

Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In International Conference on Learning Representations, 2021.

Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. arXiv preprint arXiv:1503.02531, 2015.

Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, et al. Training compute-optimal large language models. arXiv preprint arXiv:2203.15556, 2022.

Lu Hou, Zhiqi Huang, Lifeng Shang, Xin Jiang, Xiao Chen, and Qun Liu. Dynabert: Dynamic bert with adaptive width and depth. Advances in Neural Information Processing Systems, 33:9782–9793, 2020.

Ananya Harsh Jha, Dirk Groeneveld, Emma Strubell, and Iz Beltagy. Large language model distillation doesn’t need a teacher. arXiv preprint arXiv:2305.14864, 2023.

Angela H Jiang, Daniel L-K Wong, Giulio Zhou, David G Andersen, Jeffrey Dean, Gregory R Ganger, Gauri Joshi, Michael Kaminksy, Michael Kozuch, Zachary C Lipton, et al. Accelerating deep learning by focusing on the biggest losers. arXiv preprint arXiv:1910.00762, 2019.

Xiaoqi Jiao, Yichun Yin, Lifeng Shang, Xin Jiang, Xiao Chen, Linlin Li, Fang Wang, and Qun Liu. Tinybert: Distilling bert for natural language understanding. In Findings of the Association for Computational Linguistics: EMNLP 2020, pp. 4163–4174, 2020.

Jean Kaddour, Oscar Key, Piotr Nawrot, Pasquale Minervini, and Matt J Kusner. No train no gain: Revisiting efficient training algorithms for transformer-based language models. arXiv preprint arXiv:2307.06440, 2023.

Eldar Kurtic, Elias Frantar, and Dan Alistarh. Ziplm: Hardware-aware structured pruning of language models. arXiv preprint arXiv:2302.04089, 2023.

Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, Kristina Toutanova, Llion Jones, Matthew Kelcey, Ming-Wei Chang, Andrew M. Dai, Jakob Uszkoreit, Quoc Le, and Slav Petrov. Natural questions: A benchmark for question answering research. Transactions of the Association for Computational Linguistics, 2019.

François Lagunas, Ella Charlaix, Victor Sanh, and Alexander Rush. Block pruning for faster transformers. In Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, 2021.

Katherine Lee, Daphne Ippolito, Andrew Nystrom, Chiyuan Zhang, Douglas Eck, Chris Callison-Burch, and Nicholas Carlini. Deduplicating training data makes language models better. arXiv preprint arXiv:2107.06499, 2021.

Hao Li, Asim Kadav, Igor Durdanovic, Hanan Samet, and Hans Peter Graf. Pruning filters for efficient convnets. arXiv preprint arXiv:1608.08710, 2016.

Zhuohan Li, Eric Wallace, Sheng Shen, Kevin Lin, Kurt Keutzer, Dan Klein, and Joseph E Gonzalez. Train large, then compress: Rethinking model size for efficient training and inference of transformers. arXiv preprint arXiv:2002.11794, 2020.

Raymond Li et al. Starcoder: may the source be with you! arXiv preprint arXiv:2305.06161, 2023.

Jian Liu, Leyang Cui, Hanmeng Liu, Dandan Huang, Yile Wang, and Yue Zhang. Logiqa: A challenge dataset for machine reading comprehension with logical reasoning. arXiv preprint arXiv:2007.08124, 2020.

Zhuang Liu, Jianguo Li, Zhiqiang Shen, Gao Huang, Shoumeng Yan, and Changshui Zhang. Learning efficient convolutional networks through network slimming. In ICCV, 2017.

Hong Liu et al. Sophia: A scalable stochastic second-order optimizer for language model pre-training. arXiv preprint arXiv:2305.14342, 2023.

Christos Louizos, Max Welling, and Diederik P Kingma. Learning sparse neural networks through L_0 regularization. In International Conference on Learning Representations, 2018.

Jian-Hao Luo, Jianxin Wu, and Weiyao Lin. Thinet: A filter level pruning method for deep neural network compression. In ICCV, 2017.

Xinyin Ma, Gongfan Fang, and Xinchao Wang. Llm-pruner: On the structural pruning of large language models. arXiv preprint arXiv:2305.11627, 2023.

Sören Mindermann et al. Prioritized training on points that are learnable, worth learning, and not yet learnt. In ICML, 2022.

MosaicML. Introducing mpt-7b: A new standard for open-source, commercially usable llms, 2023.

OpenAI. Gpt-4 technical report. arXiv preprint arXiv:2303.08774, 2023.

Long Ouyang et al. Training language models to follow instructions with human feedback. NeurIPS, 2022.

Denis Paperno et al. The lambada dataset. arXiv preprint arXiv:1606.06094, 2016.

Guilherme Penedo et al. The refinedweb dataset for falcon llm. arXiv preprint arXiv:2306.01116, 2023.

John Platt and Alan Barr. Constrained differential optimization. In NIPS, 1987.

Jack W Rae et al. Scaling language models: Methods, analysis & insights from training gopher. arXiv preprint arXiv:2112.11446, 2021.

Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. Winogrande: An adversarial winograd schema challenge at scale. Communications of the ACM, 2021.

Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. Distilbert, a distilled version of bert: smaller, faster, cheaper and lighter. arXiv preprint arXiv:1910.01108, 2019.

Victor Sanh et al. Movement pruning: Adaptive sparsity by fine-tuning. NeurIPS, 2020.

Zhiqing Sun, Hongkun Yu, Xiaodan Song, Renjie Liu, Yiming Yang, and Denny Zhou. Mobilebert: a compact task-agnostic bert for resource-limited devices. ACL, 2020.

Mingjie Sun, Zhuang Liu, Anna Bair, and J Zico Kolter. A simple and effective pruning approach for large language models. arXiv preprint arXiv:2306.11695, 2023.

Rohan Taori et al. Stanford alpaca: An instruction-following llama model, 2023.

TogetherAI. Redpajama: An open source recipe to reproduce llama training dataset, 2023a;b.

Hugo Touvron et al. Llama: Open and efficient foundation language models. arXiv preprint arXiv:2302.13971, 2023a.

Hugo Touvron et al. Llama 2: Open foundation and fine-tuned chat models. arXiv preprint arXiv:2307.09288, 2023b.

Trieu H Trinh and Quoc V Le. A simple method for commonsense reasoning. arXiv preprint arXiv:1806.02847, 2018.

Ziheng Wang, Jeremy Wohlwend, and Tao Lei. Structured pruning of large language models. EMNLP, 2020.

Johannes Welbl, Nelson F Liu, and Matt Gardner. Crowdsourcing multiple choice science questions. arXiv preprint arXiv:1707.06209, 2017.

Wei Wen, Chunpeng Wu, Yandan Wang, Yiran Chen, and Hai Li. Learning structured sparsity in deep neural networks. NeurIPS, 2016.

Mengzhou Xia, Zexuan Zhong, and Danqi Chen. Structured pruning learns compact and accurate models. ACL, 2022.

Mengzhou Xia, Mikel Artetxe, Chunting Zhou, Xi Victoria Lin, Ramakanth Pasunuru, Danqi Chen, Luke Zettlemoyer, and Ves Stoyanov. Training trajectories of language models across scales. ACL, 2023.

Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. Smoothquant: Accurate and efficient post-training quantization for large language models. ICML, 2023.

Sang Michael Xie, Hieu Pham, Xuanyi Dong, Nan Du, Hanxiao Liu, Yifeng Lu, Percy Liang, Quoc V Le, Tengyu Ma, and Adams Wei Yu. Doremi: Optimizing data mixtures speeds up language model pretraining. arXiv preprint arXiv:2305.10429, 2023.

Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. Hellaswag: Can a machine really finish your sentence? ACL, 2019.

Peiyuan Zhang, Guangtao Zeng, Tianduo Wang, and Wei Lu. Tinyllama: An open-source small language model. arXiv preprint arXiv:2401.02385, 2024.

Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Victoria Lin, et al. Opt: Open pre-trained transformer language models. arXiv preprint arXiv:2205.01068, 2022.

Minjia Zhang and Yuxiong He. Accelerating training of transformer-based language models with progressive layer dropping. NeurIPS, 2020.

Yukun Zhu, Ryan Kiros, Rich Zemel, Ruslan Salakhutdinov, Raquel Urtasun, Antonio Torralba, and Sanja Fidler. Aligning books and movies: Towards story-like visual explanations by watching movies and reading books. ICCV, 2015.

附录 A 剪枝掩码参数化的详细阐述（A DETAILED EXPOSITION OF PARAMATERIZING PRUNING MASKS）

剪枝算法的核心思想是对模型参数施加掩码。学到二值掩码后,等价于移除对应参数。掩码用 Louizos et al. (2018) 引入的硬混凝土分布（hard concrete distribution）参数化。给定由 α 参数化的掩码变量 z，硬混凝土分布定义如下:

$$
\begin{aligned}
u &= U(0,1) \\
s &= \text{Sigmoid}\left(\frac{1}{\beta}\left(\log\frac{u}{1-u}+\log\alpha\right)\right) \\
\bar{s} &= s(\zeta-\gamma)+\gamma \\
z &= \min(1,\max(0,\bar{s}))
\end{aligned}
$$

其中 U 是均匀分布，β 是温度参数，s 是服从硬混凝土分布的松弛二值掩码，ζ 与 γ 是硬混凝土分布的边界。硬混凝土分布作为二值掩码的连续松弛，使模型能在训练中以连续方式学习二值掩码。该技巧在学习神经网络稀疏结构上的有效性已在先前研究中得到证明（Wang et al., 2020; Xia et al., 2022）。我们的实验中设 $\beta = 0.83$, $\zeta = 1.1$，$\gamma = -0.1$。

为施加稀疏约束，掩码与拉格朗日乘子 λ 一起训练，如式 (1) 所定义。剪枝后，移除与学到掩码对应的参数，以确保结果模型形状匹配目标模型。在实际实现中，我们设阈值把掩码二值化。由于采用硬混凝土分布，掩码在多数情形下通常会收敛到匹配目标模型形状的二值，从而避免不一致。但在少数掩码未精确收敛到 0 或 1 的情形下，需要把掩码变量吸收进结果模型参数中。

如第 2 节所述，我们对头、中间维、层与隐维施加掩码。对头，我们简单地用掩码乘头输出；对中间维，把掩码施加到中间输出；对层，把掩码施加到层输出；对隐维，把掩码同时施加到头输出与 MLP 输出。对输出施加掩码等价于移除对应参数。更多细节见 `composer_llama.py`。

附录 B 由缩放定律预测的参考损失（REFERENCE LOSS PREDICTED BY SCALING LAWS）

语言建模的缩放定律是模型规模 N 与数据集规模 D 的函数:

$$L(N, D) = E + \frac{A}{N^{\alpha}} + \frac{B}{D^{\beta}}$$

其中 E 刻画理想生成过程中真实语言分布的损失，A, α, B, β 是与模型规模或数据量相关的缩放因子。同一模型族中的模型通常在相同数据分布上用相同数量的 token 训练。此时，我们至少需要三个模型来估计常数 $E + \frac{B}{D^{\beta}}$、A 与 α。若模型用不同数量的 token 训练，则至少需要 5 个模型来估计 E, A, α, B, β。注意我们将分别对每个领域估计缩放因子。

LLaMA2 模型已在相同的 2T token 上训练（Touvron et al., 2023b）。我们取 LLaMA2-7B、LLaMA2-13B 与 LLaMA2-70B 检查点，在各领域验证集上评估它们，并用对应损失拟合缩放因子。由于估计缩放定律常数的数据点有限，假想 LLaMA-2.7B 模型的投影损失相对真值可能有偏。表 6 给出预测损失。评估过程耗时不到 4 个 A100 GPU 小时。

表 6：假想 LLaMA2-1.3B 与 LLaMA2-2.7B 模型的估计参考损失。

	CC	GitHub	Book	StackExchange	Wiki	ArXiv	C4
LLaMA2-1.3B	1.964	0.746	2.139	1.612	1.759	1.445	2.125
LLaMA2-2.7B	1.871	0.688	2.033	1.535	1.630	1.356	2.033

附录 C 训练细节（TRAINING DETAILS）

我们在附录 C 中给出实验所用超参数。我们用完全分片数据并行（fully sharded data parallel; Zhao et al., 2023）并行训练模型，用 FlashAttention V1（Dao et al., 2022）加速训练。我们使用余弦学习率调度器，并把学习率衰减到峰值的最低 10%。我们做了一些初步实验来确定学习掩码变量与拉格朗日乘子的峰值学习率，发现学习率 1.0 对剪枝效果良好。我们未调其他超参数。吞吐依赖于实现，我们相信通过采用更先进的近期优化（如 FlashAttention V2（Dao et al., 2022）与更新版本的 Composer（MosaicML, 2021））还可进一步提升吞吐。

表 7：训练超参数与吞吐。

	剪枝（Pruning）	继续预训练（Continued Pre-training）
Training budget	0.4B	50B
Learning rate of z, ϕ, λ	1.0	—
Learning Rate of θ	0.0001	0.0001
LR warmup ratio	10%	3%
Batch size (tokens)	131K	1M
Evaluation interval m (steps)	50	400
Steps	3,200	51,200
# GPUs	8	16
Throughput (tokens/s)	15K	145K (1.3B) / 77K (2.7B)

附录 D 模型配置（MODEL CONFIGURATIONS）

本节给出 Sheared-LLaMA 与基线模型的模型配置，见表 8。我们的设计紧密遵循 Pythia-1.4B 与 INCITE-Base-3B 的架构，但有一些细微差别。一个值得注意的差异是 Sheared-LLaMA 的中间层大小：这是它源自 LLaMA2-7B 的后果。尤其是，LLaMA2-7B 在前馈层中使用 GLU 变体（Shazeer, 2020），包含门控矩阵、上投影矩阵与下投影矩阵；相比之下，其他模型使用传统的双矩阵前馈层结构。此外，我们承认 shearing 算法必须继承源模型的头维度。我们不是按现有语言模型显式指定头数，而是把目标头数设为目标隐维除以源模型的头维度。

表 8：Sheared-LLaMA 与基线模型的模型配置。

模型	#Param	#Layers	Hidden	Intermediate	#Heads	Head Dim
OPT-1.3B	1.3B	24	2048	8192	32	64
Pythia-1.4B	1.4B	24	2048	8192	16	128
TinyLlama-1.1B	1.1B	22	2048	5632	32	64
Sheared-LLaMA-1.3B	1.3B	24	2048	5504	16	128
OPT-2.7B	2.7B	32	2560	10240	32	80
Pythia-2.8B	2.8B	32	2560	10240	32	80
INCITE-Base-3B	2.8B	32	2560	10240	32	80
OpenLLaMA-3B	2.7B	26	3200	8640	32	100

模型	#Param	#Layers	Hidden	Intermediate	#Heads	Head Dim
Sheared-LLaMA-2.7B	2.7B	32	2560	6912	20	128
LLaMA2-7B	6.7B	32	4096	11008	32	128

附录 E 指令调优（INSTRUCTION TUNING）

我们在指令调优上评估模型，并在从 ShareGPT 数据集 [10] 采样的 10,000 条指令—回复对上微调 Sheared-LLaMA 与基线模型。评估时，我们再从 ShareGPT 采样 1,000 条指令，用微调后的模型与其他基线生成回复，并用 GPT-4 作为评判者比较两条回复（Dubois et al., 2023）。我们报告相对基线模型的胜率。

指令调优训练时，指令前加上:「You are a helpful assistant. Write a response that appropriately completes the request.」。评估指令调优生成时，Wang et al. (2023a) 观察到：用 GPT 模型作裁判时，交换两条输出的呈现顺序可能改变其偏好。因此我们对每对输出交换呈现顺序比较两次，最终报告两轮的平均胜率以消除位置偏置。

[10] https://sharegpt.com。我们仅使用多轮聊天历史中的第一轮。

我们随机选取 Sheared-LLaMA-1.3B 与 Sheared-LLaMA-2.7B 对给定指令的输出，并在表 10 中展示。结果表明：经指令调优后，Sheared-LLaMA-2.7B 能始终生成长、连贯且信息丰富的输出。

表 9：指令调优的训练超参数。

	Instruction Tuning
Learning Rate of θ	5×10^{-5}
LR warmup ratio	3%
Batch size (tokens)	128
# GPUs	8

附录 F 补充实验结果（ADDITIONAL EXPERIMENT RESULTS）

F.1 继续预训练中的数据用量（DATA USAGE IN CONTINUED PRE-TRAINING）

图 6 展示训练过程中领域权重的演化以及各领域最终累积数据用量。轨迹显示：领域权重在大约 30% 的训练后稳定。相对其他领域，Wikipedia 在训练早期出现异常尖峰。其余领域的数据加载随时间呈预期的平稳、单调变化。

图 6：左：继续预训练阶段各批的数据权重；右：各领域累积数据用量。右侧柱状图对比原始 RedPajama 比例与动态批加载后的比例（如 CC 67.0%→36.1%，C4 15.0%→49.2% 等）。

表 10：Sheared-LLaMA-1.3B 与 Sheared-LLaMA-2.7B 的示例输出。表明经指令调优后，Sheared-LLaMA 能生成长、连贯且信息丰富的回复。

	内容（原文英文保留）
Prompt	Act as semiconductor industry analyst
Sheared-LLaMA-1.3B	As a semiconductor industry analyst, I would be responsible for conducting market research and analysis to help companies in the industry understand and respond to market trends, customer preferences, and competitive dynamics. This would include analyzing market data such as sales, market share, and industry trends, as well as conducting interviews with industry leaders, executives, and customers to gain insights into customer needs and preferences. I would also be responsible for developing and implementing marketing strategies to promote products and services, and for identifying and pursuing new business opportunities.
Sheared-LLaMA-2.7B	As a semiconductor industry analyst, my primary responsibility is to provide insightful and accurate analysis on the global semiconductor market. This includes evaluating the performance of individual companies, industries, and trends within the market. Here are some key areas where I would focus my analysis: (1) Market size and growth; (2) Competitive landscape; (3) Technology trends (5G, AI, IoT); (4) Industry trends; (5) Market forecasts. In addition···keep track of industry news and developments···

F.2 与 LLM-Pruner 的比较（COMPARISON TO LLM-PRUNER）

为与 LLM-Pruner 方法公平比较，我们把参数量（不含嵌入）匹配到与最终模型大致相同（1.23B），因为嵌入大小不影响推理速度。我们对来自 LLM-Pruner 与我们提出的目标化结构化剪枝的剪后模型都做继续预训练。剪枝与继续预训练的总 token 数控制为相同，并直接使用 RedPajama 数据、不应用动态批加载。我们从三方面表明目标化结构化剪枝优于 LLM-Pruner：损失轨迹、模型架构与推理速度。

就损失轨迹而言，图 7 显示：在消耗相同数据量时，我们提出的目标化结构化剪枝达到比 LLM-Pruner 更低的损失。

表 11 比较 LLM-Pruner 剪后模型与我们剪后模型的配置。LLM-Pruner 模型有非常规架构——中间层小于隐层——这很大程度上是因为该算法无法剪隐维与层，暴露了 LLM-Pruner 的局限。

就训练吞吐与推理速度而言，我们发现 Sheared-LLaMA 结构比 LLM-Pruner 模型运行更高效。我们用单块 A100 GPU 生成最多 2048 token，对 LLM-Pruner 与 Sheared-LLaMA 的模型架构做推理速度分析。如表 12 所示，我们的剪后模型架构在推理时显著更高效。此外，LLM-Pruner 的模型架构在继续预训练时引入可观开销（用 16 块 A100 80GB GPU 测量），训练吞吐约为 Sheared-LLaMA 的 60%。总体而言，相对 LLM-Pruner，我们的 Sheared-LLaMA 架构在推理与继续训练上都有更高吞吐。

总之，我们已表明：在相同参数规模下，我们的剪枝方法产出的模型困惑度（损失）更低、最终模型架构更合理、推理速度更快。我们有效证明：相对 LLM-Pruner，目标化结构化剪枝算法对大规模 LLM 剪枝更有效。

图 7：继续预训练阶段 LLM-Pruner 与 Sheared-LLaMA 的损失。注意为公平比较，我们排除动态批加载，并对两模型使用相同数据分布训练。

表 11：Pythia-1.4B、LLM-pruner (1.6B) 与 Ours (1.3B) 的模型结构。

	Layers	Heads	Head size	Intermediate size	Hidden size	Params
Pythia-1.4B	24	16	128	8192	2048	1.4B
LLM-pruner (1.6B)	32	7	128	2201	4096	1.6B
Sheared-LLaMA (1.3B)	24	16	128	5504	2048	1.3B

表 12：LLM-pruner (1.6B) 与 Sheared-LLaMA (1.3B) 的训练吞吐与生成速度。在相近参数量下，用相同 token 量（约 6B）微调后，我们的剪后模型结构困惑度更低，且训练与推理都高效得多。

	Generation Speed	Training Throughput	PPL
LLM-Pruner	43 tokens/s	83K tokens/s	7.09
Sheared-LLaMA	58 tokens/s	139K tokens/s	6.85

F.3 编码与数学推理（CODING AND MATH REASONING）

我们考察剪后模型相对其他语言模型的数学与编码能力。我们发现：包括 Sheared-LLaMA 在内，现有 3B 参数模型的数学能力仍远低于更大模型。我们也发现 Sheared-LLaMA 的编码能力落后于已知在更多代码数据上训练的模型，如 Pythia-1.4B 与 Open-LLaMA-3B-v2。Sheared-LLaMA 的编码能力很可能来自原始 LLaMA2 模型（据推测使用了更多代码数据），而我们剪枝实验中使用的代码数据很少。

表 13：GSM8K 与 HumanEval 上的评估结果，以及 ArXiv 与 GitHub 的训练百分比与 token 数。

Models	GSM8K (8) EM	HumanEval Pass@1	HumanEval Pass@5	ArXiv %	Github %	ArXiv Tokens	GitHub Tokens
LLaMA2-7B	13.7	12.8	23.8	–	–	–	–
OPT-2.7B	0.1	0.0	0.0	–	–	–	–
Pythia-2.8B	1.7	5.1	14.6	9.0%	7.6%	26.9	22.8
INCITE-Base-3B	1.8	4.3	4.9	2%	4.5%	16.0	36.0
Open-LLaMA-3B-v1	2.5	0.0	1.2	2%	4.5%	20.0	45.0
Open-LLaMA-3B-v2	2.7	10.4	20.1	–	–	–	–
Sheared-LLaMA-2.7B (Source)	2.7	3.7	5.5	0.7%	0.4%	0.3	0.2
Sheared-LLaMA-2.7B (Scaling)	2.4	4.9	9.2	1.0%	0.8%	0.5	0.4

F.4 Scaling Reference vs. Source Reference

本节比较 Sheared-LLaMA 在动态批加载中分别用 scaling reference 与 source reference 训练时的表现。Scaling reference 用缩放定律预测的损失作为参考损失；source reference 用源模型损失作为参考损失。尽管两种方法都能高效训练模型，scaling reference 始终取得略好的下游表现。

图 8：Sheared-LLaMA-1.3B 在 scaling reference 与 source reference 下的平均下游准确率随训练 token 数变化。

F.5 剪枝 Pythia 模型（PRUNING PYTHIA MODELS）

在方法初期开发时，我们在 Pythia（Biderman et al., 2023）上实验了更小规模模型——Pythia 是一系列开源模型，有开源训练数据，规模从 70M 到 13B。我们取 Pythia-440M 模型，剪到 160M 参数，并用 Pythia 模型的训练数据（Gao et al., 2020）继续预训练。具体地，剪枝用 0.4B token，剪后模型继续预训练用 33B token（32,000 步）。表 14 显示：剪后模型达到比原模型更低的困惑度，继续预训练进一步改进表现。值得注意的是：以极少算力（10B token），剪枝 Pythia-410M 模型大致达到与从零预训练 Pythia-160M 相同的表现；加入更多 token 进一步提升表现。

表 14：Pythia-160M 与 Sheared-Pythia 的零样本表现。

	Training Tokens	Performance
Pythia-160M	300B	43.56
Sheared-Pythia	(300B) + 10B	43.51
Sheared-Pythia	(300B) + 33B	45.78

此外，我们把 Sheared-Pythia-160M 与用相同 token 量继续预训练 Pythia-160M 相比较。由图 9 可见：继续预训练 Pythia-160M 起初表现更好，但 Sheared-Pythia-160M 学得更快，最终超过继续预训练 Pythia-160M 的表现。这些是我们在该特定设定下看到的一些非常初步的结果。

图 9：继续预训练 Pythia-160M 与 Sheared-Pythia-160M 的下游表现。Sheared-Pythia-160M 最终超过继续预训练 Pythia-160M。

我们认为：基于先前工作（Li et al., 2020）的结论——剪更大模型再压缩比直接训小模型表现更好，因为更大模型更易优化——剪枝更大模型的收益会更显著。但更细致的分析留待未来工作。

F.6 从 LLaMA1 vs LLaMA2 剪枝（PRUNING FROM LLAMA1 VS LLAMA2）

本节比较从 LLaMA1 与 LLaMA2 剪枝的表现。两模型都展现强下游任务表现，尽管从 LLaMA2 剪枝不出意外地始终更优。但值得注意的是，两者表现差距并不很大。

图 10：对 1.3B，在动态加载下从 LLaMA1 与 LLaMA2 剪枝的比较。

F.7 与进一步继续预训练 INCITE-Base-3B 的比较（COMPARISON TO FURTHER CONTINUAL PRE-TRAINING INCITE-BASE-3B）

我们通过比较继续预训练的 INCITE-Base-3B 与 Sheared-LLaMA-2.7B 的表现，考察剪枝是否比同等规模现有 LLM 为继续预训练提供更好初始化。损失曲线见图 11，下游表现见图 12。INCITE-Base-3B 起初任务准确率更高，但训练后趋于平台；而 Sheared-LLaMA 快速改进并超过 INCITE-Base-3B，表明来自强基座的剪后模型是更好的初始化。[11]

[11] 当现有小模型相对剪枝源模型已有竞争力时，小模型可能比剪后模型提供更好起点。直觉上，源模型与小模型表现差距越大，剪后模型优势越大。

图 11：继续预训练 INCITE-3B 与我们剪后的 LLaMA 模型的损失。两模型均约 2.7B 参数。
图 12：继续预训练 Sheared-LLaMA vs INCITE-Base-3B 的平均下游表现。

我们对继续预训练 INCITE-Base-3B 使用学习率 1×10^{-5}，调度器在前 3% 训练步把学习率预热到 1×10^{-5}，并遵循余弦衰减。事后看，按近期研究（Gupta et al., 2023），我们继续预训练 INCITE-Base-3B 的方式可能并非最优。

F.8 剪枝时排除简单领域（EXCLUDING EASY DOMAINS DURING PRUNING）

在项目开发过程中，我们探索了一个简单直观的想法，以应对剪枝与继续预训练时损失下降速率不均。具体地，我们在剪枝时排除 GitHub、StackExchange 与 ArXiv 数据，因为这三个领域损失下降最快。我们用 C4、CC、Wiki 与 Books 的组合数据集（启发式比例分别为 40%、40%、10%、10%）把 LLaMA1-13B 剪到 7B；然后在包含剪枝时被排除领域的 RedPajama 数据集上继续预训练剪后模型。

结果显示：不用这三个领域数据剪枝时，各领域困惑度差异更均匀。但在用 RedPajama 全部七个领域数据继续预训练后，损失差距扩大，GitHub 上的差远小于 C4 等领域。这些结果表明：仅在剪枝阶段排除易恢复领域，并不能从本质上解决各领域损失差的失衡。

这组实验促使我们开发动态批加载，作为应对剪枝与继续预训练中领域特定损失差距的更有效、更有原则的方法。

表 15：用 40% CC、40% C4、10% Books、10% Wikipedia 的组合把 LLaMA1-13B 剪到 7B。给出源模型（LLaMA1-13B）的领域损失、剪后模型损失以及继续预训练后的损失。相对目标模型（LLaMA1-7B）的损失差在剪枝后更平衡，但在用全部领域继续预训练后更分散。

	CC	GitHub	Book	StackExchange	Wikipedia	ArXiv	C4
LLaMA1-13B	1.7585	0.6673	1.9499	1.4207	1.4331	1.3855	1.8619
LLaMA1-7B	1.8366	0.7108	2.0322	1.5112	1.5291	1.4340	1.9331
Pruned model (w/o three domains)	2.1849	1.0971	2.3726	1.9080	2.1151	1.7542	2.3187
diff from LLaMA1-7B	0.3483	0.3863	0.3404	0.3968	0.5860	0.3202	0.3857
Continued Pretraining (w RP)	1.8344	0.6325	2.0984	1.4542	1.4549	1.4460	2.0395
diff from LLaMA1-7B	−0.0022	−0.0783	0.0661	−0.0570	−0.0743	0.0120	0.1064

F.9 推理速度分析（INFERENCE SPEED ANALYSIS）

本节分析不同剪枝方法的推理速度，包括以下模型：

- 源模型，即 LLaMA2-7B；
- Sheared-LLaMA-1.3B 与 Sheared-LLaMA-2.7B；
- Wanda 剪枝（Sun et al., 2023）：一次性把 LLM 剪成半结构化 2:4 与 4:8 稀疏模式；
- LLM-Pruner（Ma et al., 2023）：产出与 Sheared-LLaMA 相同非嵌入参数量的模型。

我们用一块 A100 GPU 测试所有这些剪后模型的生成速度（tokens/秒），批大小为 1，最多生成 2048 token。结果见表 16。Sheared-LLaMA 的速度优于 LLM-Pruner，很大程度上因其结果架构更优化。如表 11 所示，LLM-Pruner 产出中间层小于隐层的模型结构，这有悖于 Transformer 设计中中间层至少为隐层 3–4 倍的惯例。

Wanda 型半结构化剪枝相对源模型也能加速推理，但不如小稠密模型快，且灵活性更低——因为仅在稀疏度为 50% 时推理加速才可行。

表 16：不同剪枝方法的推理速度（tokens/s）。

Model	Throughput
7B LLaMA-7B	37
1.3B LLM Pruner	41
2.7B LLM Pruner	40
1.3B Sheared-LLaMA	62
2.7B Sheared-LLaMA	47
50% sparsity Wanda (2:4)	42（相对 7B 设定下的半结构化结果；原文按规模分组）
50% sparsity Wanda (4:8)	42

> 译者注：原文表 16 将 Wanda 结果放在「50% sparsity」行、对应较大规模列；上表按原文数值整理，具体列对齐请对照 PDF。

附录 G 常见问题（FREQUENTLY ASKED QUESTIONS）

本节回答关于我们工作的常见问题。

▷ 能否公平地说 Sheared-LLaMA 模型仅用 50B token 即可产出，即便源模型（LLaMA2）在 2T token 上训练?

在论文投稿时，尚无在 1.3B 与 2.7B 规模上充分训练达 2T token 的模型可供公平比较。然而，近期发布的、在 3T token 上训练的 TinyLlama-1.1B 提供了合适参照。我们观察到：作为基座模型时，TinyLlama-1.1B 在下游基准上的表现与 Sheared-LLaMA-1.3B 相当；Wang et al. (2023b) 中也有类似观察。考虑到 TinyLlama-1.1B 用 3T token 训练，超过 Sheared-LLaMA-1.3B 所用预训练与剪枝总量（源模型预训练 2T，剪枝与继续训练 50.4B），我们把这视为有力证据：剪枝可能是训练中等规模 LM 本质上更高效、更有效的途径。

▷ 动态批加载与 Doremi（Xie et al., 2023）有何不同?

动态批加载与 Doremi 共享同一原则：用指数上升算法根据模型损失调整各领域数据分布。但动态批加载提供更灵活、更不复杂、可应用于多种场景的方法。

Doremi 遵循多步过程:(1)训练参考模型;(2)训练代理模型,通过根据代理模型损失调整比例来估计各领域数据比例;(3)用估计的数据分布训练最终模型。相比之下,动态批加载可直接应用于任何模型,无需参考模型或代理模型。动态批加载先基于固定评估集导出参考损失——该参考损失可用缩放定律估计,或直接用源模型的评估损失。训练中,根据周期测量的评估损失实时调整数据比例。动态批加载过程可无缝接入标准预训练流水线,因为评估损失计算高效、不引入显著开销。尽管动态批加载依赖固定评估集——可能无法完全代表模型在整个数据集上的表现——该问题可通过在训练中周期性更新评估集来缓解。

▷ **当有多种源模型规模可用时,如何选择用于剪枝的源模型规模?**

确定剪枝的最优源模型规模很有挑战。不过我们可以做一个思想实验:把每个参数视为均匀的「信息单元」。例如,若 7B 参数的源模型用 2T token 训练,可假设每个参数大约携带 $2\mathrm{T}/7\mathrm{B} = 285$ 个 token 的信息(假设信息在参数间均匀分布)。把该模型随机剪到 1.3B 参数时,总信息量减为 $1.3\mathrm{B} \times 285 = 0.37\mathrm{T}$ token。相比之下,若把同样用 2T token 训练的 13B 模型剪到 1.3B 参数,总信息量减为 $1.3\mathrm{B} \times (2\mathrm{T}/13\mathrm{B}) = 0.2\mathrm{T}$ token。尽管该估计粗糙,它暗示:尤其当源模型用相同数量 token 训练时,从更大模型剪枝可能效果较差。必须指出这是简化估计,参数间信息均匀分布的假设实践中未必成立;而且结构化剪枝过程本身显然打破了该假设。尽管如此,这一思想实验提供了源模型规模如何影响剪枝有效性的总体直觉。

译者说明:附录 A–G 已按 arXiv PDF(https://arxiv.org/abs/2310.06694)全文译出;表 6–16、图 6–12 说明与 FAQ 均已覆盖。正文部分未改动。案例/示例输出中的英文按原文保留。